Command Pattern

The Command pattern is a behavioral design pattern that encapsulates a request as an object, allowing users to parameterize clients with queues, requests, and operations. It transforms a request into an independent object with all the request's details. This separation enables more versatile operations such as queuing, logging, undoing actions, and delaying execution.

Consider an online shopping platform, which is a well-known experience for many people. You may be browsing, placing things in your cart, deciding to cancel your order, or perhaps changing your mind and exchanging an item for another. Without the proper system in place, these actions might turn into a chaotic mess for both the user and the underlying software architecture.

Applying the command pattern to it converts every action into a command object, such as adding, removing, or modifying an order. This object contains all of the components required to complete the task.

Real-World Example

A practical example of a Command Pattern is a smart home application. This application allows you to control different electronic devices, like TVs, lights, and air conditioners, with your phone.

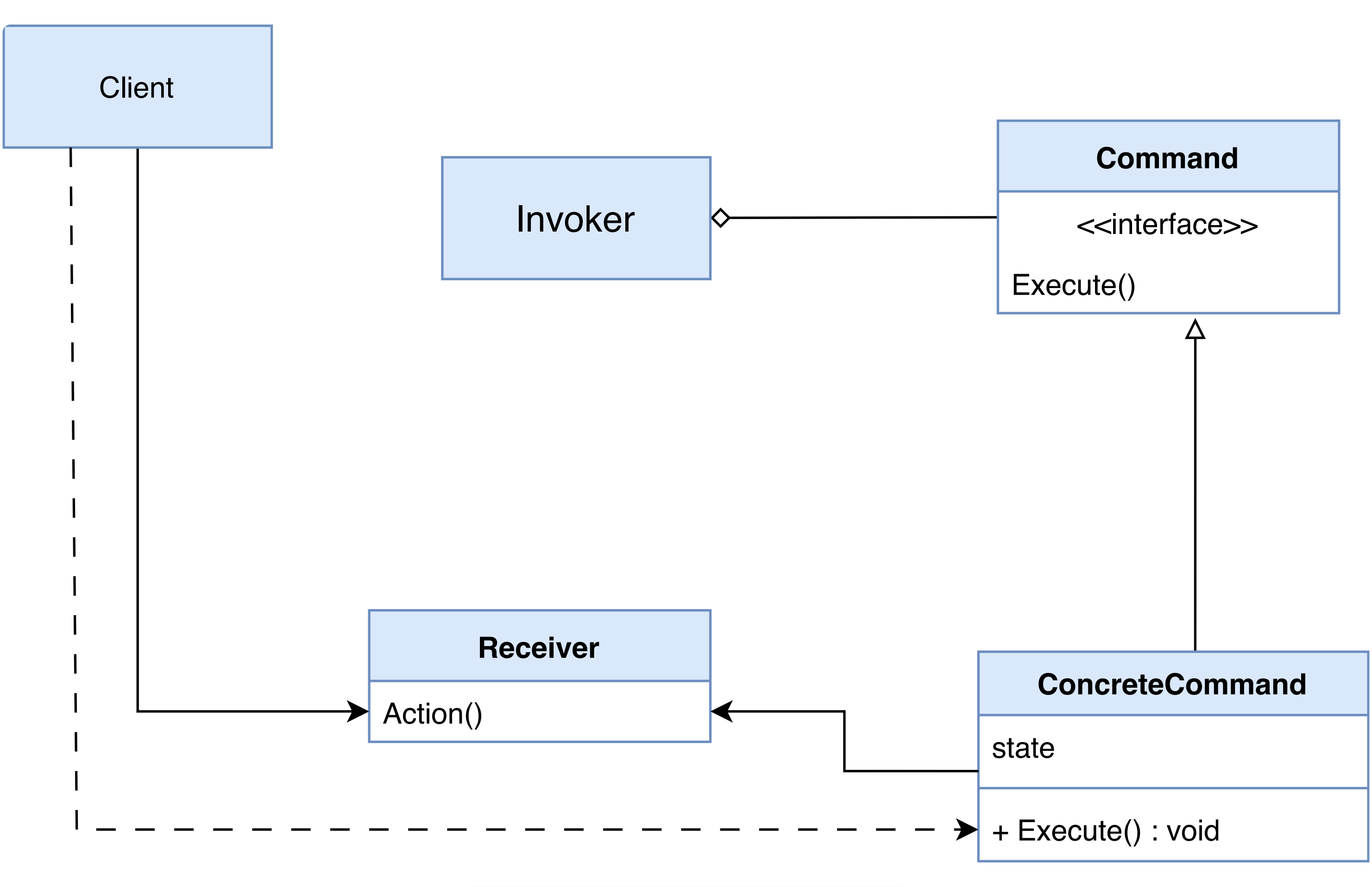
The app presents a simple, user-friendly interface with buttons for every device or action. Each button acts as a command. Pressing a button sends a command to the corresponding device. Using the command pattern, it's easy to add some new devices to this app, hence increasing the flexibility of the application.

Command Pattern - Smart Home Application Example

Structure of Command Pattern

The command pattern consists of the following key components:

- **Command**: An interface that declares the execute method is known as a command interface. It serves as a template for all concrete command classes.
- **ConcreteCommand Classes**: These classes define the binding between an action and a Receiver object and implement the Command interface. An execute method on every ConcreteCommand class calls the relevant action on the Receiver.
- **Invoker**: This class has a command stored in it, and at some point, it asks the command to execute the request. The Invoker doesn't know about the concrete implementation of the command; it just knows about the command interface.
- **Receiver**: This class can handle the tasks involved in completing a request. Any class that can perform the action the command needs can act as a Receiver.
- **Client**: This class instantiates a ConcreteCommand object and assigns it a receiver. It can also establish the association between the command and the receiver by assigning the command to the Invoker.



Command Pattern - Class Diagram

Implementation of Command Pattern

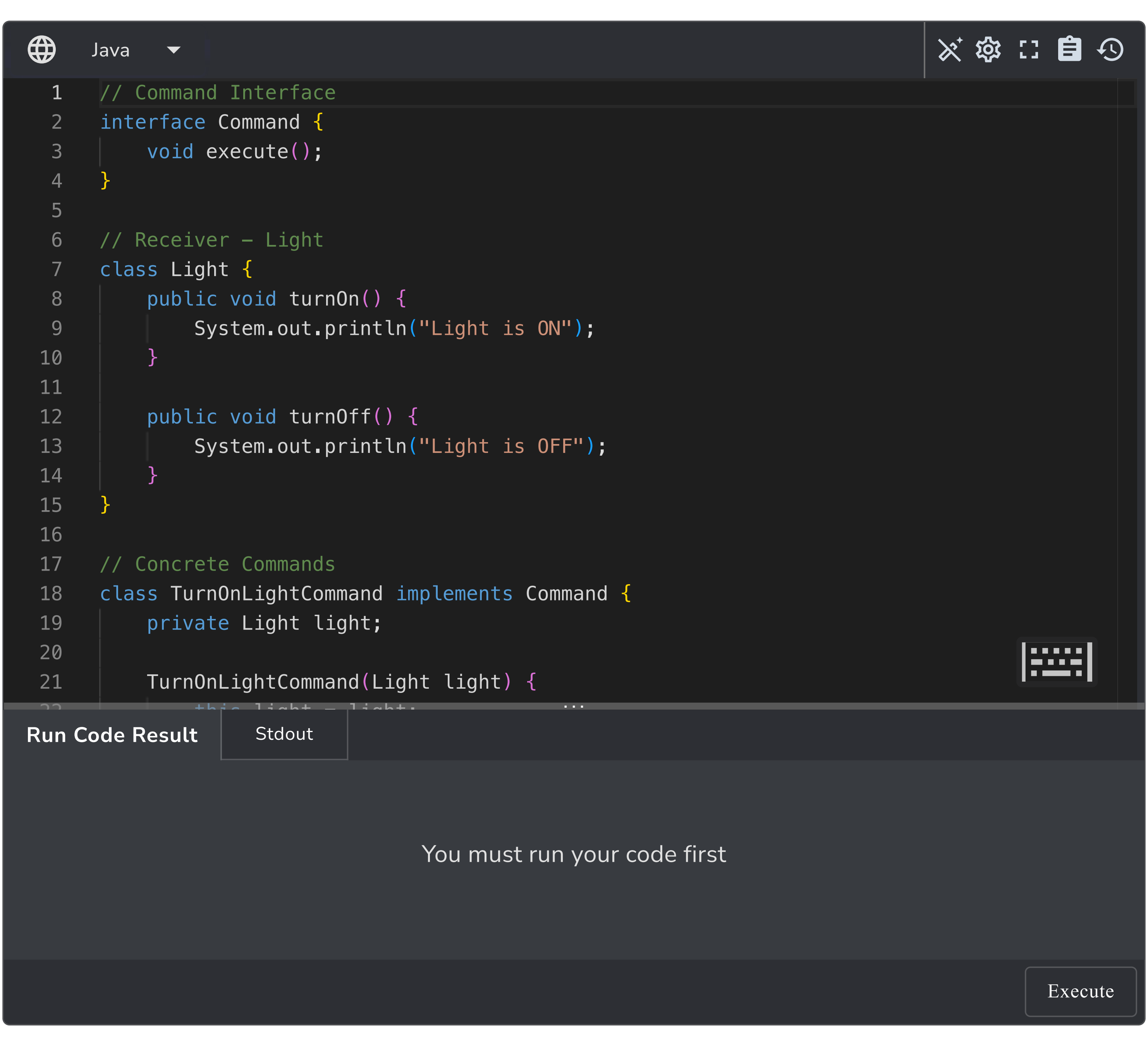
This example demonstrates a simple light control system using the Command pattern. The system can turn a light on and off using commands.

Pseudocode

```
Interface Command {
    Method execute()
}

Class Light {
    Method turnOn() {
        Print "Light is ON"
    }

    Method turnOff() {
        Print "Light is OFF"
    }
}

Class TurnOnLightCommand implements Command {
    Private light: Light

    Constructor(light: Light) {
        this.light = light
    }

    Method execute() {
        light.turnOn()
    }
}

Class TurnOffLightCommand implements Command {
    Private light: Light

    Constructor(light: Light) {
        this.light = light
    }

    Method execute() {
        light.turnOff()
    }
}

Class RemoteControl {
    Private command: Command

    Method setCommand(command: Command) {
        this.command = command
    }

    Method pressButton() {
        command.execute()
    }
}

// Client Code
Main {
    light = new Light()
    turnOn = new TurnOnLightCommand(light)
    turnOff = new TurnOffLightCommand(light)

    remote = new RemoteControl()
    remote.setCommand(turnOn)
    remote.pressButton()

    remote.setCommand(turnOff)
    remote.pressButton()
}
```

- **Command Interface**: Defines a basic execute method for all commands.
- **Light** (**Receiver**): Represents a light with methods turnOn and turnOff.
- **Concrete Commands**: TurnOnLightCommand and TurnOffLightCommand implement the Command interface. Each class holds a reference to a Light object and defines the execute method to turn the light on or off, respectively.
- **RemoteControl** (**Invoker**): Holds a command and triggers it when the pressButton method is called.
- **Client Code**: Sets up the system by creating a Light object, command objects, and a RemoteControl. It then uses the remote to execute the commands.

Implementation



```java
// Command Interface
interface Command {
    void execute();
}

// Receiver - Light
class Light {
    public void turnOn() {
        System.out.println("Light is ON");
    }

    public void turnOff() {
        System.out.println("Light is OFF");
    }
}

// Concrete Commands
class TurnOnLightCommand implements Command {
    private Light light;

    TurnOnLightCommand(Light light) {
```

You must run your code first

Application of Command Pattern

- **Graphical User Interfaces** (**GUIs**): The Command pattern works best in GUI applications when different controls, such as buttons, menus, or shortcuts initiate actions. It is simpler to handle user interactions in a scalable and maintainable manner when each action is encapsulated as a command.
- **Undo/Redo Actions**: The Command pattern plays a crucial role in enabling undo and redo features in programs such as text editors, image editors, and other programs that need to keep track of and reverse user actions.
- **Task Scheduling and Queuing**: The Command pattern can represent tasks as command objects in scenarios where they must be scheduled, queued, and executed later. This method is frequently used in thread pools, job queues, and batch processing.
- **Game programming**: The Command pattern can be used to handle AI commands, player actions, and input controls in game development.
- **Networking**: It is easier to manage different kinds of network operations and responses when network requests are encapsulated using the Command pattern.
- **Workflow Systems**: Workflow or pipeline systems allow for the easy modification or extension of processes by encapsulating each step or action as a command. This gives the workflows a clear structure.

Pros and Cons

Pros	Cons
Decoupling of Actions and Actors Decouples the objects that send commands from the ones that perform them.	**Complexity** It can increase the number of classes and complexity of the system.
Ease of Extending Commands New commands can be added without modifying the existing commands.	**Overhead** Each action might require a new command class, that increases overhead.
Simplifying Complex Operations By breaking the complex operations into simpler commands, operations can be simplified.	**Indirect Execution** Commands introduce an additional layer of abstraction, which can complicate understanding and debugging.
Support for Undo/Redo Facilitates implementing undo/redo mechanisms.	

The command pattern is a very useful tool in your design pattern toolbox. It's like possessing a magic wand that neatly condenses complex requests into small, doable objects. This leads to a world of possibilities, including the ability to undo functions, log actions, and much more, and make your code cleaner and more organized.

← Previous
Chain Of Responsibility Pattern
Next →
Iterator Pattern
☐ Mark as Completed